

Documento 04: Hoja de Ruta Técnica para el Desarrollo Backend en PHP

1. Introducción y Propósito del Documento

Este manual está diseñado para guiar paso a paso al equipo de desarrollo en la transición del Front-End maquetado en HTML/JS hacia un **Backend dinámico, seguro y profesional programado en PHP (Hypertext Preprocessor - Preprocesador de Hipertexto)** conectado a la base de datos **MySQL**.

El documento asume una explicación didáctica desde cero, detallando cómo fluyen los datos entre el navegador y el servidor, cómo estructurar el código bajo el patrón **MVC (Modelo-Vista-Controlador)** y cómo proteger la aplicación contra amenazas de seguridad comunes.

2. Glosario de Conceptos y Siglas de Backend

- **PHP (Hypertext Preprocessor - Preprocesador de Hipertexto)**: Lenguaje de programación de código abierto ejecutado en el servidor web (como Apache o Nginx). A diferencia de JavaScript (que se ejecuta en el navegador del cliente), PHP procesa datos en el servidor, consulta la base de datos y genera el HTML final que se envía al usuario.
- **HTTP (Hypertext Transfer Protocol - Protocolo de Transferencia de Hipertexto)**: Protocolo de comunicación que permite el intercambio de información en la web mediante peticiones (*Requests*) del navegador y respuestas (*Responses*) del servidor.
- **MVC (Model-View-Controller - Modelo-Vista-Controlador)**: Patrón de arquitectura de software que separa la aplicación en tres capas fundamentales:
 1. **Modelo (Model)**: Gestiona los datos, las consultas a la base de datos y las reglas del negocio.
 2. **Vista (View)**: Es la interfaz gráfica (las pantallas HTML y CSS) que se le muestra al usuario.
 3. **Controlador (Controller)**: El intermediario o "director de orquesta" que recibe la petición del usuario, llama al Modelo para obtener o guardar datos y selecciona qué Vista mostrar.
- **PDO (PHP Data Objects - Objetos de Datos de PHP)**: Capa de abstracción oficial de PHP para interactuar con bases de datos. Proporciona una interfaz orientada a objetos consistente y permite el uso de **Sentencias Preparadas**.
- **Sentencias Preparadas (Prepared Statements)**: Técnica de programación donde la consulta SQL se envía primero al motor de la base de datos con marcadores de posición (`?` o `:nombre`), y luego los datos del usuario se envían por separado. Esto hace **imposible la Inyección SQL**, ya que los datos nunca se interpretan como código ejecutable.
- **CRUD (Create, Read, Update, Delete - Crear, Leer, Actualizar, Borrar)**: Las cuatro operaciones básicas y universales del almacenamiento persistente de datos.
- **SQL Injection (Inyección SQL)**: Vulnerabilidad crítica de seguridad donde un atacante introduce código SQL malicioso en un campo de formulario para alterar las consultas de la base de datos (por ejemplo para robar contraseñas o borrar tablas).
- **XSS (Cross-Site Scripting - Secuencias de Comandos en Sitios Cruzados)**: Vulnerabilidad donde un atacante inyecta código JavaScript malicioso en una página vista por otros usuarios. Se previene en PHP escapando las salidas con `htmlspecialchars()`.

- **CSRF (Cross-Site Request Forgery - Falsificación de Petición en Sitios Cruzados):** Ataque donde un sitio malicioso engaña al navegador de un usuario autenticado para que ejecute acciones no deseadas en la aplicación (por ejemplo borrar un torneo). Se previene mediante un **Token CSRF** secreto generado por la sesión.
- **Front Controller (Controlador Frontal):** Patrón de diseño web donde todas las peticiones del sitio web ingresan a través de un único archivo central (`index.php`), el cual inicializa la configuración, valida la sesión y delega la ejecución al controlador correspondiente.

3. Estructura de Directorios Recomendada para el Backend

Para migrar de forma ordenada y limpia, organizaremos el código fuente de la siguiente manera:

```
SGDM/
├── codigo_fuente/
│   ├── config/                <-- Parámetros de configuración del
│   │                               sistema
│   ├── database.php           <-- Credenciales de conexión a MySQL
│   └── global.php             <-- Constantes del sistema (Rutas,
URLs base)
│   ├── controladores/        <-- Capa de Controladores (Lógica de
flujo)
│   │   ├── AuthControlador.php <-- Login, Registro, Logout, Recuperar
│   │   ├── UsuarioControlador.php <-- CRUD de usuarios y roles
│   │   └── TorneoControlador.php <-- Creación, configuración y llaves
Clave
│   │   ├── EquipoControlador.php <-- Gestión de equipos, miembros y
de torneos
│   │   └── ResultadoControlador.php <-- Carga y validación de marcadores
solicitudes
│   │   └── modelos/           <-- Capa de Modelos (Acceso a Base de
Datos)
│   │       ├── Conexion.php    <-- Conexión PDO Singleton
│   │       └── Usuario.php      <-- Consultas sobre tablas 'usuarios'
y perfiles
│   │       ├── Torneo.php       <-- Consultas sobre 'torneos',
│   │       └── Equipo.php        <-- Consultas sobre 'equipos' y
'encuentros', 'posiciones'
│   │           ├── 'equipo_miembros'
│   │           └── vistas/       <-- Capa de Vistas (Plantillas PHP con
HTML)
│   │       ├── admin/          <-- Vistas administrativas
│   │       │   ├── layout/      <-- Partes comunes reutilizables
│   │       │   │   ├── header.php <-- Cabecera HTML y menú superior
│   │       │   │   ├── sidebar.php <-- Barra lateral de navegación
│   │       │   │   └── footer.php  <-- Pie de página y scripts
│   │       │   └── dashboard.php <-- Vista de métricas
│   │       │       ├── torneos/
│   │       │       │   ├── lista.php
│   │       │       │   └── crear.php
│   │       └── auth/            <-- Pantallas de autenticación
```

```

|   |   |   | jugador/           <-- Pantallas del competidor
|   |   |   | organizador/       <-- Pantallas del organizador
|   |   |   | publico/           <-- Portal público
|   |   |   | publico/           <-- Archivos estáticos accesibles
|   |   |   | directamente
|   |   |   | css/
|   |   |   | js/
|   |   |   | img/
|   |   |   | index.php          <-- FRONT CONTROLLER (Punto de entrada
único)

```

🔧 4. Paso a Paso: Guía de Implementación para la Próxima Semana

A continuación se presentan los bloques de código fundacionales listos para implementar:

PASO 1: Conexión Segura a la Base de Datos con PDO

Creamos el archivo `codigo_fuente/modelos/Conexion.php` aplicando el patrón **Singleton (Instancia Única)** para no saturar el servidor abriendo múltiples conexiones innecesarias:

```

<?php
// codigo_fuente/modelos/Conexion.php

class Conexion {
    private static ?PDO $instancia = null;

    // Parámetros de conexión a la base de datos
    private const HOST = 'localhost';
    private const DB_NAME = 'ascend';
    private const USER = 'root';
    private const PASSWORD = '';
    private const CHARSET = 'utf8mb4';

    // Constructor privado para impedir instanciación directa con 'new'
    private function __construct() {}

    public static function obtenerConexion(): PDO {
        if (self::$instancia === null) {
            $dsn = "mysql:host=" . self::HOST . ";dbname=" . self::DB_NAME .
";charset=" . self::CHARSET;

            $opciones = [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, // Lanza
excepciones ante errores SQL
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, //
Devuelve resultados como arreglos asociativos
                PDO::ATTR_EMULATE_PREPARES => false, // Utiliza
sentencias preparadas nativas del motor
            ];

```

```

        try {
            self::$instancia = new PDO($dsn, self::USER, self::PASSWORD,
            $opciones);
        } catch (PDOException $e) {
            // Registro de error seguro (no exponer contraseñas al usuario
            final)
            error_log("Error de conexión a la Base de Datos: " . $e-
            >getMessage());
            die("Error de comunicación con el servidor. Intente más tarde.");
        }
    }
    return self::$instancia;
}
}

```

PASO 2: El Front Controller Central ([index.php](#))

Reemplaza la lógica temporal de [mockup-router.js](#) por un enrutador en el servidor:

```

<?php
// codigo_fuente/publico/index.php

// 1. Iniciar sesión de forma segura
session_start([
    'cookie_httponly' => true, // Impide que JavaScript lea la cookie de sesión
    (Protección XSS)
    'cookie_samesite' => 'Lax', // Protección básica contra CSRF
]);

// 2. Cargar dependencias base
require_once __DIR__ . '/../modelos/Conexion.php';

// 3. Capturar controlador y acción solicitada desde la URL (ej: index.php?
c=torneo&a=crear)
$controladorNombre = $_GET['c'] ?? 'admin';
$accionNombre      = $_GET['a'] ?? 'dashboard';

// 4. Mapear al archivo del controlador
$controladorClase = ucfirst($controladorNombre) . 'Controlador';
$controladorRuta  = __DIR__ . '/../controladores/{$controladorClase}.php";

if (file_exists($controladorRuta)) {
    require_once $controladorRuta;
    $controlador = new $controladorClase();

    if (method_exists($controlador, $accionNombre)) {
        $controlador->$accionNombre();
    } else {
        http_response_code(404);
    }
}

```

```

        echo "Acción no encontrada en el sistema.";
    }
} else {
    http_response_code(404);
    echo "Página o controlador no encontrado.";
}

```

PASO 3: Modelo de Datos para Usuarios y Herencia de Perfiles

Creamos `codigo_fuente/modelos/Usuario.php` implementando consultas preparadas seguras:

```

<?php
// codigo_fuente/modelos/Usuario.php

require_once __DIR__ . '/Conexion.php';

class Usuario {
    private PDO $db;

    public function __construct() {
        $this->db = Conexion::obtenerConexion();
    }

    /**
     * Autentica un usuario verificando correo y contraseña encriptada.
     */
    public function autentificar(string $email, string $contrasena): ?array {
        $sql = "SELECT id, email, contrasena_hash, nombre_completo, rol_id,
esta_activo
                FROM usuarios
                WHERE email = :email LIMIT 1";

        $stmt = $this->db->prepare($sql);
        $stmt->execute([':email' => $email]);
        $usuario = $stmt->fetch();

        // Verificar si existe y si la contraseña coincide con el hash BCrypt
        if ($usuario && password_verify($contrasena, $usuario['contrasena_hash']))
        {
            if (!$usuario['esta_activo']) {
                return null; // Cuenta suspendida
            }
            return $usuario;
        }
        return null;
    }

    /**
     * Registra un nuevo usuario y crea su perfil según el rol (Herencia 1:1 en
3FN).

```

```

    */
    public function registrar(array $datos): int {
        try {
            $this->db->beginTransaction();

            // 1. Insertar en tabla padre 'usuarios'
            $hash = password_hash($datos['contrasena'], PASSWORD_BCRYPT, ['cost'
=> 10]);

            $sqlPadre = "INSERT INTO usuarios (email, contrasena_hash,
nombre_completo, telefono, rol_id)
                        VALUES (:email, :hash, :nombre, :telefono, :rol_id)";

            $stmt = $this->db->prepare($sqlPadre);
            $stmt->execute([
                ':email'      => $datos['email'],
                ':hash'       => $hash,
                ':nombre'     => $datos['nombre_completo'],
                ':telefono'   => $datos['telefono'] ?? null,
                ':rol_id'     => $datos['rol_id']
            ]);

            $usuarioId = (int)$this->db->lastInsertId();

            // 2. Insertar en tabla hija correspondiente según el rol
            if ($datos['rol_id'] == 2) {
                // Rol 2: Organizador -> Inserta en 'perfiles_organizadores'
                $sqlHija = "INSERT INTO perfiles_organizadores (usuario_id,
nombre_organizacion, localidad)
                        VALUES (:usuario_id, :organizacion, :localidad)";
                $stmtHija = $this->db->prepare($sqlHija);
                $stmtHija->execute([
                    ':usuario_id' => $usuarioId,
                    ':organizacion' => $datos['organizacion'] ?? 'Organizador
Independiente',
                    ':localidad'   => $datos['localidad'] ?? null
                ]);
            } elseif ($datos['rol_id'] == 3) {
                // Rol 3: Jugador -> Inserta en 'perfiles_jugadores'
                $sqlHija = "INSERT INTO perfiles_jugadores (usuario_id,
apodo_gamertag, pais)
                        VALUES (:usuario_id, :apodo, :pais)";
                $stmtHija = $this->db->prepare($sqlHija);
                $stmtHija->execute([
                    ':usuario_id' => $usuarioId,
                    ':apodo'      => $datos['apodo'] ?? $datos['nombre_completo'],
                    ':pais'       => $datos['pais'] ?? 'Uruguay'
                ]);
            }

            $this->db->commit();
            return $usuarioId;
        } catch (Exception $e) {
            $this->db->rollBack();
            error_log("Error al registrar usuario: " . $e->getMessage());
        }
    }
}

```

```

        throw $e;
    }
}
}

```

PASO 4: Cómo Transformar las Maquetas HTML en Vistas PHP Reutilizables

Para evitar duplicar código, separamos las maquetas en componentes modulares:

1. **codigo_fuente/vistas/admin/layout/header.php**: Contiene el `<!DOCTYPE html>`, la etiqueta `<head>`, las hojas de estilo CSS y la barra superior con el usuario logueado:

```

<!-- Cabecera dinámica -->
<header class="cabecera-principal">
    <h1>ASCEND</h1>
    <div class="perfil-usuario">
        <span><?= htmlspecialchars($_SESSION['usuario_nombre'] ??
'Administrador') ?></span>
    </div>
</header>

```

2. **codigo_fuente/vistas/admin/layout/sidebar.php**: La barra lateral con enlaces dinámicos en PHP:

```

<aside class="menu-lateral">
    <ul>
        <li><a href="index.php?c=admin&a=dashboard"><img alt="Dashboard icon" data-bbox="658 565 678 585"/> Dashboard</a></li>
        <li><a href="index.php?c=torneo&a=lista"><img alt="Torneos icon" data-bbox="628 585 648 605"/> Torneos</a></li>
        <li><a href="index.php?c=usuario&a=lista"><img alt="Usuarios icon" data-bbox="628 605 648 625"/> Usuarios</a></li>
        <li><a href="index.php?c=auth&a=logout"><img alt="Cerrar Sesión icon" data-bbox="628 625 648 645"/> Cerrar Sesión</a></li>
    </ul>
</aside>

```

3. **codigo_fuente/vistas/admin/torneos/lista.php**: El contenido específico de la pantalla que se combina con el layout:

```

<?php require_once __DIR__ . '/../layout/header.php'; ?>
<?php require_once __DIR__ . '/../layout/sidebar.php'; ?>

<main class="area-contenido">
    <h2>Listado de Torneos</h2>
    <table class="tabla-estandar">
        <thead>
            <tr>
                <th>Torneo</th>
                <th>Disciplina</th>
                <th>Estado</th>

```

```

        <th>Acciones</th>
    </tr>
</thead>
<tbody>
    <?php foreach ($torneos as $torneo): ?>
        <tr>
            <td><?php htmlspecialchars($torneo['nombre']) ?></td>
            <td><?php htmlspecialchars($torneo['disciplina_nombre']) ?>
        </td>
            <td><span class="etiqueta-estado"><?php
htmlspecialchars($torneo['estado']) ?></span></td>
            <td>
                <a href="index.php?c=torneo&a=editar&id=<?php
$torneo['id'] ?>" class="boton-accion">Editar</a>
            </td>
        </tr>
    <?php endforeach; ?>
</tbody>
</table>
</main>

<?php require_once __DIR__ . '/../layout/footer.php'; ?>

```

5. Medidas de Seguridad Obligatorias en PHP

Durante el desarrollo de la próxima semana, se deberán respetar siempre estas tres directivas de seguridad en todos los módulos:

1. Protección Anti-Inyección SQL:

- **Prohibido:** Concatenar variables directamente en la consulta SQL ("SELECT * FROM usuarios WHERE email = '\$email'").
- **Obligatorio:** Usar siempre sentencias preparadas con PDO (\$stmt = \$db->prepare("SELECT * FROM usuarios WHERE email = :email"); \$stmt->execute([':email' => \$email]));

2. Protección Anti-XSS (Cross-Site Scripting):

- **Obligatorio:** Al imprimir cualquier variable proveniente de la base de datos o de un formulario en el HTML, envolverla con htmlspecialchars(\$variable, ENT_QUOTES, 'UTF-8').

3. Control de Acceso y Sesión (RBAC):

- En la cabecera de cada controlador administrativo, verificar que exista la sesión y que el rol tenga los privilegios necesarios:

```

if (!isset($_SESSION['usuario_id']) || $_SESSION['rol_id'] !== 1) {
    header("Location: index.php?c=auth&a=login");
    exit();
}

```